Demo Task

AI Software Engineer Internship

Read This Carefully Before Starting

This assignment is designed to evaluate how you think, build, and execute as an engineer. We are not expecting perfect production systems. We are evaluating your ability to take ownership, make decisions independently, and build reliable software under ambiguity.

You may use AI tools such as ChatGPT, Claude, Cursor, Copilot, or any other developer tools during implementation. However, you must fully understand your code and be able to explain your architecture decisions during review.

Step 1 — Choose Your Role

Choose ONLY ONE role:

Role 1 - Frontend Engineer

Focus areas:

- UI architecture
- frontend systems
- product experience
- state management
- reusable components

Role 2- Backend Engineer

Focus areas:

- APIs
- schema design
- validation systems
- backend architecture
- data handling

Role 3- Full Stack Engineer

Focus areas:

- end-to-end systems
- frontend + backend integration
- deployment
- architecture quality

Step 2 — Choose ONE Track

Choose ONLY ONE track:

- Track A → AI App Generator
- Track B → College Discovery Platform
- Track C → Compensation Intelligence System

Your evaluation will happen only based on your chosen role + chosen track.

Mandatory Tech Stack

Please use the following stack unless absolutely necessary otherwise.

Frontend

- Next.js
- React
- TypeScript
- TailwindCSS

Backend

- Node.js
- TypeScript
- Next.js API Routes or NestJS

Database

- PostgreSQL
- Prisma ORM

Preferred Deployment

- Vercel
- Railway
- Render
- Cloudflare
- Neon

📤 Submission

Submit via **Google Form** : https://forms.gle/YHpUKzBBWwzrSyjTA

- **Live URL** (working product)
- **GitHub repository URL**
- **Loom video (5-10 min)** explaining architecture, decisions taken, edge cases handled, tradeoffs

---

TRACK A — AI App Generator

Reference - https://base44.com/

Objective

Build a metadata-driven application runtime that converts JSON configuration into a working application.

Your system should dynamically generate:

- frontend UI
- APIs
- database structure
- workflows

The JSON configuration may contain:

- missing fields
- invalid values
- unknown components
- inconsistent schemas

Your system should handle these cases gracefully without breaking.

Frontend Engineer Expectations

Build a frontend system capable of dynamically rendering forms, dashboards, tables, and layouts directly from configuration.

Your application should support:

- reusable components
- loading states
- error states
- responsive layouts
- extensible component architecture

The frontend should never crash because of invalid configuration. Unknown components or broken fields should fail gracefully.

We will evaluate:

- frontend architecture
- rendering logic
- state management
- extensibility
- reliability under broken configs

Backend Engineer Expectations

Build a backend runtime capable of:

- dynamic API generation
- CRUD execution
- validation handling
- schema management
- reliable error handling

Your APIs should support:

- optional fields
- malformed input
- schema mismatches

- validation failures

Authentication should support user-scoped data access.

We will evaluate:

- API architecture
- schema design
- validation systems
- reliability
- edge-case handling

Full Stack Engineer Expectations

Build the complete end-to-end platform including:

- frontend rendering engine
- backend runtime
- database architecture
- authentication
- deployment

Additionally, implement ANY THREE:

- CSV import
- notifications
- multi-language support
- multi-auth login
- GitHub export
- workflow automation
- mobile/PWA support

These features should integrate properly into your architecture and not exist as isolated implementations.

TRACK B — College Discovery Platform

References:

https://www.careers360.com/

https://collegedunia.com/

Objective

Build a production-grade MVP for a college discovery and decision-making platform.

You are NOT expected to build a complete marketplace.

Choose ANY 3–4 features and execute them extremely well.

Available Features

1. College Listing + Search

Build searchable college listings with filters and pagination/infinite scroll.

Each listing should include:

- college name
- location
- fees
- rating

---

2. College Detail Page

Build a detailed college page containing:

- overview
- courses
- placements
- reviews

---

3. Compare Colleges

Build a side-by-side comparison experience for 2–3 colleges.

Comparison should include:

- fees

- placements
- ratings
- location

---

4. Predictor Tool

Allow users to input:

- exam
- rank

Return recommended colleges based on logic or dataset-driven matching.

---

5. Q&A / Discussion

Build a simple discussion system where users can:

- ask questions
- answer questions
- browse discussions

---

6. Authentication + Saved Items

Allow users to:

- login/signup
- save colleges
- save comparisons

Frontend Engineer Expectations

Focus on:

- search UX
- filtering experience
- comparison flows
- responsive design

- reusable UI systems
- frontend performance
- API state handling

We care more about usability and architecture quality than animations or visual polish.

Backend Engineer Expectations

Focus on:

- search APIs
- filtering systems
- pagination
- database modeling
- validation systems
- REST API quality

You may use mock or generated datasets, but all data must come from the database and APIs.

Avoid hardcoded frontend-only implementations.

Full Stack Engineer Expectations

Build a complete product including:

- frontend experience
- backend APIs
- authentication
- database integration
- deployment

The final product should feel cohesive, functional, and production-oriented.

TRACK C — Compensation Intelligence System

References:

https://www.levels.fyi/ **(primary reference)**

https://www.ambitionbox.com/

Objective

Build a compensation intelligence platform focused on structured and comparable salary data.

This is NOT a salary listing website.

The system should help users compare compensation intelligently using:

- levels
- roles
- location
- compensation structure

Core principle:
Levels matter more than job titles.

Mandatory Research

Before implementation, analyze:

- Levels.fyi
- 6figr
- AmbitionBox
- Glassdoor

Submit:

1. Key observations
2. Feature comparison sheet

Example format:

| Feature | Levels.fyi | 6figr | AmbitionBox | Glassdoor | Build? |

Frontend Engineer Expectations

Build:

- searchable salary tables
- filtering UX
- comparison interfaces
- company pages
- compensation visualizations

Your application should support:

- sorting
- filtering
- responsive layouts
- structured comparison flows

We will evaluate:

- usability
- frontend architecture
- structured data presentation
- performance with large datasets

Backend Engineer Expectations

Build APIs for:

- salary ingestion
- filtering
- company aggregation
- compensation comparison

Your backend should:

- normalize company names
- calculate total compensation
- reject invalid data
- handle duplicate entries
- default missing bonus/stock to 0

We will evaluate:

- schema quality
- normalization logic
- API architecture
- validation systems
- reliability

Full Stack Engineer Expectations

Build the complete platform including:

- frontend
- backend
- database
- comparison workflows
- deployment

The final system should feel reliable, structured, and scalable.

We care more about:

- clean architecture
- reliable systems
- extensibility
- execution quality

than:

- excessive features
- visual effects
- unnecessary complexity

Important -

AI-assisted coding tools such as ChatGPT, Claude, Cursor, Antigravity, and GitHub Copilot are allowed. However, fully generated no-code/low-code platforms are NOT allowed.

Do NOT use:

- **Replit AI app generation**
- **Lovable**
- **Bolt.new**

- **Base44**
- **Firebase Studio generated apps**
- **Google AI Studio generated apps**
- **similar prompt-to-app generators**

We are evaluating engineering ability, architecture decisions, and implementation understanding. You must understand your code and explain your decisions clearly.

⚠️ Important Note

This task is intentionally challenging.

We have designed it to create a steep learning curve and to identify people who can operate with high ownership and high agency.

You are not expected to know everything beforehand.
But you are expected to:

- **figure things out independently**
- **navigate ambiguity**
- **make decisions without waiting for instructions**
- **push through technical and product challenges**

This is not a typical internship assignment.
It reflects the kind of problems you will work on here.

If you're someone who enjoys:

- **building from scratch**
- **solving unclear problems**
- **taking full ownership**

you will find this exciting.

If you prefer structured, step-by-step tasks, this may not be the right fit.